

8 de julho de 2026 Pesquisa Publicação

Separando sinal de ruído em avaliações de programação

Por meio de uma auditoria detalhada, encontramos problemas generalizados nas tarefas do SWE-Bench Pro e estimamos que cerca de 30% delas estejam quebradas.

[▶ Ouvir artigo](#) 9:42[🔗 Compartilhar](#)

Medir com precisão as capacidades dos nossos modelos é importante para decisões sólidas de implantação e segurança, incluindo decisões no âmbito do [Preparedness Framework](#) da OpenAI. A cada lançamento de modelo, relatamos resultados em diversos benchmarks externos e internos para acompanhar o progresso dos modelos. Quando as avaliações têm falhas que afetam os resultados, elas podem produzir uma compreensão falsa das capacidades, distorcendo análises de segurança e afetando prioridades de pesquisa.

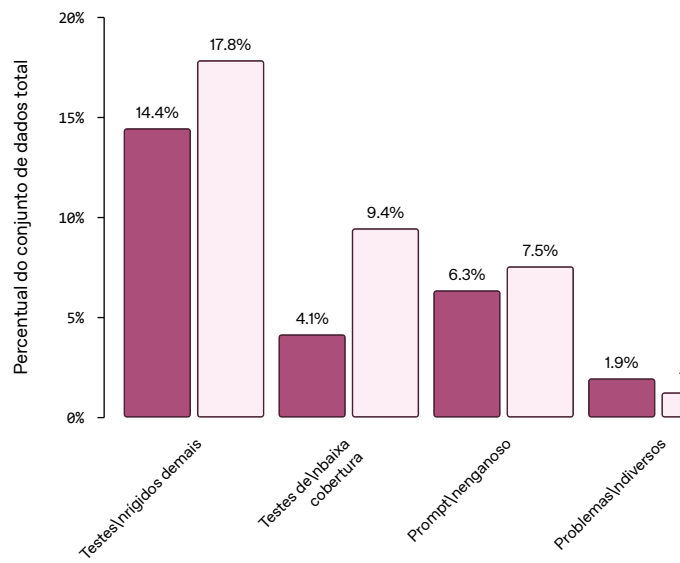
Nós [investigamos recentemente](#) como um dos benchmarks de programação mais usados, o SWE-bench Verified, tinha problemas fundamentais de design e contaminação, e constatamos que a avaliação já não fornecia um sinal significativo sobre capacidades de desenvolvimento de software. Na ocasião, incentivamos a comunidade em geral a migrar para o SWE-Bench Pro.

O [SWE-Bench Pro](#) foi criado para aprimorar o SWE-bench Verified ao testar modelos em horizontes mais longos e tarefas de programação mais realistas, a fim de acompanhar melhor capacidades de programação agêntica. Como no SWE-bench Verified, as tarefas são obtidas programaticamente a partir do histórico de alterações de recursos em um conjunto de repositórios públicos e privados. Os modelos precisam implementar uma solução que passe em novos testes de um recurso sem quebrar funcionalidades existentes. Na divisão pública de 731 tarefas, modelos de fronteira melhoraram de uma taxa de aprovação de 23,3% para 80,3% em oito meses.

Desde então, realizamos uma auditoria semelhante no SWE-Bench Pro, revisando o conjunto de dados com um pipeline de análise de pontos de dados. O pipeline revisou tentativas do modelo na tarefa, metadados da tarefa e rastros de falha para sinalizar prováveis falhas de avaliação. Cada tarefa sinalizada foi então avaliada por várias passagens de investigadores-agentes e revisada de forma

Parcela do conjunto de dados sinalizada por tipo de problema

● Revisão por agente com supervisão humana ○ Anotações humanas



Encontramos evidências de problemas críticos em uma parte significativa do conjunto de dados. Nosso pipeline de análise de pontos de dados sinalizou 200 tarefas problemáticas (27,4%), enquanto a campanha de anotação humana identificou 249 (34,1%).

Os problemas se concentraram principalmente em quatro categorias:

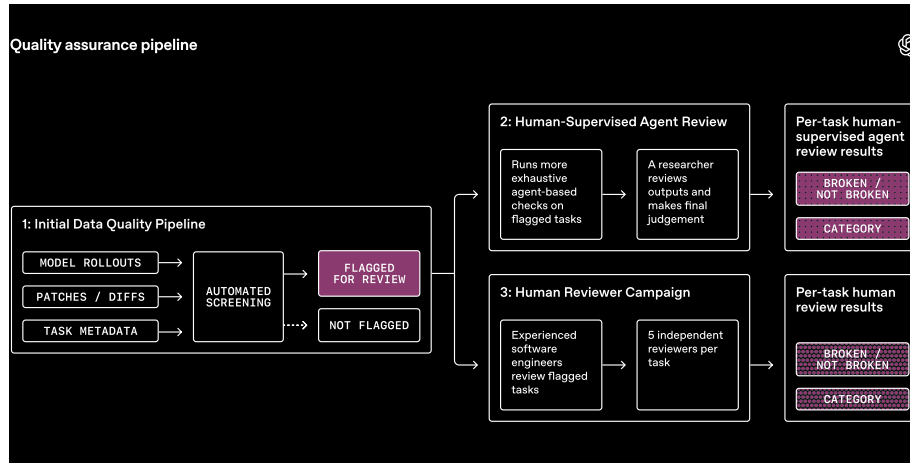
- *Testes excessivamente rígidos*¹ impõem detalhes específicos de implementação não especificados no prompt, invalidando muitas submissões funcionalmente corretas.
- *Prompts subespecificados*² omitem requisitos que testes ocultos impõem e que não são razoavelmente inferíveis.
- *Testes de baixa cobertura* verificam de forma insuficiente o recurso solicitado, permitindo que correções incompletas sejam aprovadas.
- Um *prompt enganoso* direciona os modelos para o comportamento errado ou contradiz o que os testes exigem.

Nossas conclusões apontam para a dificuldade de curar benchmarks difíceis, mas justos, e para a utilidade crescente de agentes em verificações escaláveis de qualidade de dados. À luz desses resultados, estimamos que cerca de 30% das tarefas do SWE-bench Pro têm problemas e recomendamos que desenvolvedores de modelos examinem os resultados com cuidado.

Metodologia

OpenAI

reais dos modelos, e que sucessos em tarefas reflitam soluções completas e válidas para os requisitos do prompt. Para verificar a qualidade dos dados usados na avaliação, criamos um pipeline de garantia de qualidade para avaliar se cada ponto de dados reflete com precisão as capacidades dos modelos.



Um pipeline inicial de qualidade de dados sinaliza problemas para revisão. Validamos com uma auditoria mais aprofundada, assistida por agentes, das tarefas sinalizadas e uma campanha de anotação humana com engenheiros experientes.

Um filtro automatizado inicial revisa as instruções dadas ao modelo, as tentativas do modelo de resolver a tarefa e os testes usados para avaliar essas tentativas, sinalizando exemplos provavelmente quebrados ou problemáticos. Esse filtro sinalizou 286 tarefas potencialmente problemáticas. Em seguida, conduzimos uma revisão mais aprofundada desse subconjunto de duas maneiras: uma revisão por agente com supervisão humana, que realiza verificações extensas com agentes investigadores e um julgamento humano final; e uma campanha de anotação humana com desenvolvedores de software experientes.

Revisão por agente com supervisão humana

Cada problema sinalizado é auditado com agentes investigadores baseados no Codex, que receberam acesso ao repositório e ao ambiente da tarefa. Isso os ajuda a distinguir ambiguidades razoáveis da tarefa, que muitas vezes podem ser resolvidas estudando código próximo e convenções do repositório, de uma subespecificação real. O agente pode executar testes, inspecionar arquivos no repositório e investigar tentativas do modelo e seus modos de falha comuns na tarefa. Após várias repetições independentes dessas auditorias mais profundas, um pesquisador revisou os resumos, fez um julgamento final e rotulou os prováveis problemas.

Campanha de anotação humana

Em paralelo, conduzimos uma campanha de anotação humana no subconjunto sinalizado. Trabalhamos com engenheiros de software experientes, treinados nos objetivos do benchmark, na taxonomia de problemas e em casos-limite antes de revisar as tarefas. Cada tarefa foi revisada por cinco engenheiros.

Os revisores formaram um julgamento independente a partir do enunciado visível do problema, dos casos de teste e da solução de referência verdadeira (conhecida como gold patch) antes de usar a análise do pipeline ou a transcrição como contexto de apoio. Em seguida, os revisores atribuíram um rótulo e uma classificação de gravidade com base em evidências concretas, e escalaram divergências ou casos de baixa confiança para nova revisão.

Os revisores humanos tiveram maior probabilidade do que os agentes investigadores de marcar tarefas como problemáticas. Também houve alguma divergência sobre categorias entre os dois caminhos de revisão, mas em nenhuma tarefa sinalizada “não problemática” foi o rótulo humano mais comum. Das categorias sinalizadas pelo pipeline de agentes, os julgamentos dos revisores coincidiram em 74% dos casos.

Em comparação com o pipeline de agentes, os revisores humanos também foram mais propensos a selecionar vários rótulos para uma tarefa, indicando que consideraram as tarefas problemáticas de várias maneiras ou que elas não se encaixavam claramente em uma única categoria. Isso sugere que o pipeline com agente e revisor resultou em uma rotulagem conservadora: ele capturou os mesmos modos amplos de falha identificados por humanos, mas subcontou casos em que os revisores viram problemas adicionais ou sobrepostos. A maior diferença apareceu em testes de baixa cobertura, que os humanos selecionaram como o problema mais comum em 9,4% do benchmark, contra 4,1% no pipeline de agentes.

Modos de falha

Prompt enganoso

Testes excessivamente rígidos

Prompt subespecificado

Testes de baixa cobertura

Testes excessivamente rígidos tornam as tarefas extremamente difíceis ou impossíveis para um modelo resolver, pois testam detalhes de implementação, como nomes de variáveis e métodos privados ou strings literais de erro, que não são especificados no prompt nem razoavelmente inferíveis, de modo que implementações funcionalmente corretas falhariam nos testes.

OpenAI

```

15         m := message{senderCtx: ctx}
16         Expect(b.shouldSend(m, c)).To(BeTrue())
17     })
18     It("does not send message for same username")
19     {
20         ctx = request.WithClientUniqueId(ctx, "jane")
21         ctx = request.WithUsername(ctx, "jane")
22         m := message{senderCtx: ctx}
23         Expect(b.shouldSend(m, c)).To(BeFalse())
24     })
25     It("does not send message for different username")
26     {
27         ctx = request.WithClientUniqueId(ctx, "john")
28         ctx = request.WithUsername(ctx, "john")
29         m := message{senderCtx: ctx}
30         Expect(b.shouldSend(m, c)).To(BeFalse())
31     })
32 })

```

Discussão

Os problemas que identificamos, somados a casos semelhantes no [SWE-bench Verified](#), destacam a importância de verificar benchmarks com rigor. Issues e pull requests de repositórios open-source foram originalmente criados para colaboração humana, muitas vezes por meio de longas idas e vindas entre mantenedores e colaboradores. Como resultado, descrições de problemas, código mesclado e testes unitários nem sempre se alinham para formar tarefas limpas e isoladas capazes de avaliar modelos de forma confiável. Em particular, testes incluídos em pull requests podem ser excessivamente rígidos porque são escritos para validar uma mudança específica, e não para definir um padrão independente de implementação para resolver a tarefa.

Ao mesmo tempo, falhas de avaliação são hoje mais fáceis de detectar do que seriam mesmo há pouco tempo. À medida que as capacidades dos modelos melhoram, podemos usar esses modelos para inspecionar prompts, testes, patches, rastros e casos-limite com muito mais profundidade e consistência, ajudando a revelar problemas de benchmark que antes eram caros ou impraticáveis de encontrar em escala.

Esperamos que a comunidade de avaliação mais ampla desenvolva novos benchmarks criados por desenvolvedores de software experientes especificamente para testar capacidades dos modelos. Essa abordagem pode preservar o alto padrão e o realismo que queremos ao medir capacidades dos modelos, além de permitir melhor supervisão humana ao longo de todo o processo. Diante dos problemas revelados nesta análise, retiramos nossa recomendação anterior de adotar o SWE-Bench Pro.

Em última análise, uma avaliação deve fornecer um sinal significativo por meio de benchmarks difíceis de manipular, fáceis de confiar e que reflitam genuinamente a capacidade ou o

OpenAI

acompanhamos precisam ser válidas e informativas.

2026 Software e engenharia

Autor

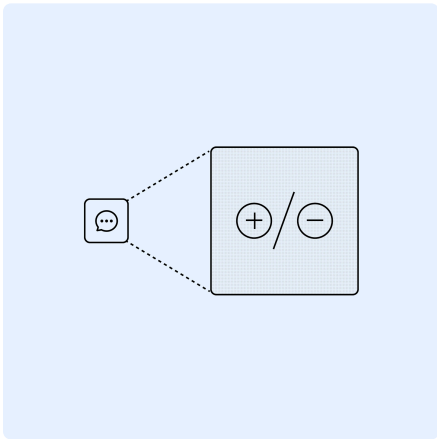
OpenAI

Notas de rodapé

- 1 Nós anteriormente nos referíamos a esta categoria como testes estreitos. ↩
- 2 Anteriormente, nos referíamos a esta categoria como testes amplos. ↩

Continuar lendo

Ver tudo



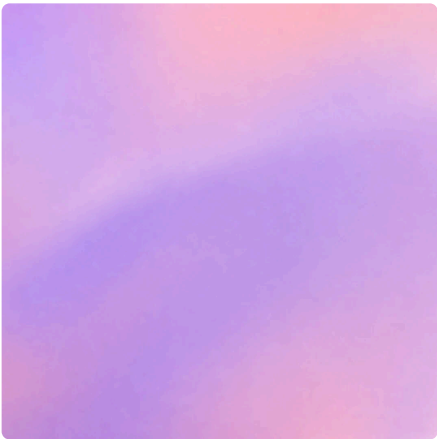
Apresentamos o MentalHealthBench

Publicação 23 de set. de 2026



Apresentamos o GPT-6 Sol e o Luna

Produto 22 de set. de 2026



Nossa estrutura para relatar desalinhamento de modelos

Pesquisa 16 de set. de 2026

Pesquisa

Índice de Pesquisa

Visão Geral de Pesquisa

Pesquisa Econômica

Últimos Avanços

GPT-6

GPT-5.6

GPT-5.5

GPT-5.4

Produtos

ChatGPT ↗

ChatGPT Business ↗

ChatGPT Enterprise ↗

ChatGPT para Educação ↗

Codex

Notas de lançamento

Plataforma de API

Visão geral

Business

Visão geral

Soluções

Recursos

Plugins

Histórias de Usuários

Rede de parceiros

Fale com Vendas

Empresa

Sobre Nós

Nosso Estatuto

Carreiras

Noticias

Suporte

Central de Ajuda ↗

Mais

Histórias

Academia

Supply Co.

Transmissões ao vivo

Podcast

RSS

Termos e Políticas

Termos de uso

OpenAI

- Segurança da implantação ↗
- Segurança e Privacidade
- Confiança e Transparência

- Modelos Abertos
- Documentos ↗
- Recursos ↗
- Fórum de Desenvolvedores ↗

